Lezione 4

Lezione 4 — Adminer e MySQL con Docker Compose

In questa esercitazione avviamo due servizi con Docker Compose: Adminer, che fornisce una semplice interfaccia web, e MySQL, che contiene il database. Docker Compose crea automaticamente una rete privata e permette ai due servizi di comunicare usando i loro nomi.

Obiettivi

- descrivere più container in un file Compose;
- avviare e fermare più servizi con un solo comando;
- usare la rete predefinita creata da Docker Compose;
- collegare Adminer al database MySQL usando il nome del servizio;
- creare una tabella e inserire alcuni dati;
- confrontare Docker Compose con l'avvio manuale dei container.

Prerequisiti

- Docker Desktop oppure Docker Engine avviato;
- un browser web;
- un terminale aperto nella directory `lezione4`.

Verificare l'installazione:

```
docker --version
docker compose version
```

1. Esaminare i file dell'esempio

La directory contiene il file di configurazione di Docker Compose:

```
lezione4/
└── compose.yaml
```

Il file definisce due servizi:

- `db`, basato sull'immagine `mysql:8.0`;
- `adminer`, basato sull'immagine `adminer`.

Compose crea automaticamente una rete per il progetto. Per questo motivo non è necessario creare una rete con `docker network create`.

Controllare la configurazione senza avviare i servizi:

```
docker compose config
```

2. Avviare i servizi

```
docker compose up -d
```

L'opzione `-d` avvia i servizi in background. Compose scarica le immagini se non sono già presenti e crea la rete del progetto.

Controllare lo stato dei servizi:

```
docker compose ps
```

MySQL potrebbe impiegare alcuni secondi per completare l'avvio. Osservare i log con:

```
docker compose logs -f db
```

Quando nei log compare un messaggio simile a `ready for connections`, premere `Ctrl+C`. I servizi continueranno a funzionare.

Per osservare i log di tutti i servizi:

```
docker compose logs
```

3. Collegarsi al database dal browser

Aprire http://localhost:8080.

Compilare il modulo di Adminer con questi valori:

```
Sistema: MySQL
Server: db
Utente: root
Password: rootpass
Database: scuola
```

Il campo **Server** deve contenere `db`, cioè il nome del servizio MySQL nel file `compose.yaml`. Non bisogna usare `localhost`: dentro Adminer, `localhost` indicherebbe il container Adminer stesso.

Docker Compose permette ai servizi della stessa rete di risolversi tramite il nome del servizio. Adminer quindi raggiunge MySQL usando `db`.

4. Creare una tabella

Dopo il login, scegliere **SQL command** e inserire:

```sql
CREATE TABLE studenti (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    corso VARCHAR(100) NOT NULL
);

INSERT INTO studenti (nome, corso) VALUES
    ('Anna Rossi', 'Docker'),
    ('Luca Bianchi', 'Kubernetes');

SELECT * FROM studenti;
```

Eseguire il comando e verificare che vengano visualizzati i due studenti.

5. Osservare i servizi e la rete

Elencare i servizi gestiti da Compose:

```
docker compose ps
```

I container creati dal progetto possono essere osservati anche con:

```
docker ps
```

Per individuare la rete creata da Compose:

```
docker network ls
```

Il nome della rete contiene normalmente il nome della directory del progetto, seguito da `_default` oppure da `-default`, a seconda della versione di Compose. Per ispezionare la rete, usare il nome visualizzato dal comando precedente:

```
docker network inspect lezione4_default
```

Nella sezione `Containers` devono comparire sia il database sia Adminer.

Il concetto importante è:

> `db` e `adminer` sono due servizi descritti nello stesso file. Compose crea la rete e avvia i container; Adminer raggiunge MySQL usando il nome `db`.

La porta `8080` è pubblicata sul computer solo per permettere al browser di raggiungere Adminer. La comunicazione tra Adminer e MySQL avviene direttamente sulla rete interna di Compose.

6. Confronto con la lezione 3

Nella lezione 3 era necessario eseguire separatamente:

```
docker network create lezione3-rete
docker run ... mysql:8.0
docker run ... adminer
```

In questa lezione la configurazione dei due servizi è raccolta in `compose.yaml` e il ciclo di vita dell'esercitazione è gestito con pochi comandi:

```
docker compose up -d
docker compose ps
docker compose logs
docker compose down
```

La rete non viene creata manualmente e non è necessario ricordare i singoli comandi `docker run`.

7. Pulizia finale

Terminata l'esercitazione, fermare e rimuovere i container e la rete del progetto:

```
docker compose down
```

Questo esempio non usa un volume: rimuovendo il container MySQL si eliminano anche i dati creati durante l'esercitazione.

Riepilogo

- `compose.yaml` descrive più servizi in un unico file;
- `docker compose up` crea la rete e avvia i servizi;
- i servizi sulla stessa rete possono raggiungersi usando il loro nome;
- `localhost` indica sempre il container o il computer dal quale viene usato;
- la porta pubblicata serve per collegare il browser al container Adminer;
- `docker compose down` rimuove i container e la rete del progetto;
- senza un volume, il database è temporaneo e non persistente.